COURSE NAME: ***SOFTWARE ENGINEERING AND PROJECT MANAGEMENT***

COURSE CODE: BCS501

Module 2 - Requirements Engineering • Requirements Modeling: Scenarios, Information and Analysis Classes • Requirements Modeling Strategies

---

Chapter 5: Requirements Engineering

5.1 Requirements Engineering – Overview

Requirements engineering is the broad umbrella of tasks that help a software team understand what problem it is solving before design or construction begins. It builds a bridge between the needs and expectations of stakeholders and the design and construction of the software system. Pressman identifies seven distinct tasks within requirements engineering: inception, elicitation, elaboration, negotiation, specification, validation, and requirements management.

- Inception – stakeholders establish the basic problem, the people who want a solution, the nature of the solution desired, and how the software and system fit into business/customer needs.
- Elicitation – finding out from stakeholders what the objectives for the system are, what is to be accomplished, and how it fits into business needs.
- Elaboration – the information from inception and elicitation is expanded and refined; an analysis model that identifies data, function, and behavioural requirements is created.
- Negotiation – requirements are prioritised, risks are identified, and rough estimates of resources are agreed with stakeholders through reconciliation of conflicts.
- Specification – the requirements are represented in a form that can later be reviewed and evaluated – a written document, a set of models, a prototype, or a combination of these.
- Validation – the work products of requirements engineering are examined for consistency, omissions, and ambiguity.
- Requirements management – the set of activities that help the team identify, control, and track requirements and changes at any time as the project proceeds.

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

5.2 Establishing the Groundwork

5.2.1 Identifying Stakeholders

A stakeholder is anyone who benefits, directly or indirectly, from the system being developed – business operations managers, product managers, marketing people, internal/external customers, end users, consultants, product engineers, software engineers, support/maintenance engineers, and others. Every stakeholder has a different view of the system and, whether realistic or not, unarticulated expectations.

5.2.2 Recognizing Multiple Viewpoints

Because many stakeholders exist, requirements will be examined from many points of view. Each point of view is likely to result in a different set of requirements and conflicts may occur. Collaboration is required to reconcile these to produce a consistent set of requirements that satisfies most stakeholders.

5.2.3 Working Toward Collaboration

Requirements engineering is a collaborative negotiation process. Stakeholders present their own views of requirements and these views may or may not be compatible with those of other stakeholders. The goal is for stakeholders to negotiate on a set of requirements by playing different roles and seeking a common vision.

5.2.4 Asking the First Questions

Context-free questions focus on the customer, overall goals, and benefits, e.g., 'Who is behind the request for this work?', 'Who will use the solution?', 'What will be the economic benefit of a successful solution?', 'Is there another source for the solution needed?' Other questions enable the analyst to gain a better understanding of the problem, and let the customer voice perceptions about a solution.

5.3 Eliciting Requirements

Requirements elicitation (also called requirements gathering) combines elements of problem solving, elaboration, negotiation, and specification. Stakeholders and engineers work together to identify the problem, propose elements of the solution, negotiate approaches, and specify a preliminary set of solution requirements.

5.3.1 Collaborative Requirements Gathering

Many different approaches have been proposed for collaborative requirements gathering. Each makes use of a slightly different scenario, but all apply some variation of the following basic guidelines:

- Meetings are conducted and attended by both software engineers and other stakeholders.
- Rules for preparation and participation are established.

- An agenda is suggested – yet flexible enough to include the ideas of stakeholders.
- A 'facilitator' (customer, developer, or outsider) controls the meeting.
- A 'definition mechanism' (worksheets, flip charts, wall stickers, electronic board) is used.

The goal is to identify the problem, propose elements of the solution, negotiate different approaches, and specify a preliminary set of requirements in an atmosphere conducive to the accomplishment of the goal.

5.3.2 Quality Function Deployment (QFD)

QFD is a quality management technique that translates the needs of the customer into technical requirements for software. QFD identifies three types of requirements:

- Normal requirements – objectives and goals stated for a product/system during meetings with the customer. Satisfying most normal requirements makes the customer happy (e.g., requested types of graphical display).
- Expected requirements – implicit to the product/system and so fundamental that the customer does not explicitly state them; their absence is a significant cause for dissatisfaction (e.g., ease of human/machine interaction).
- Exciting requirements – features that go beyond customer expectations and prove to be very satisfying when present, but the customer does not ask for them since they are unaware of their potential (e.g., a novel feature that adds unexpected value).

QFD uses customer interviews and observation, surveys, and examination of historical data as raw information for the requirements-gathering activity; this data is then translated into a table of requirements – a function deployment table, which value engineering.

5.3.3 Usage Scenarios (Use Cases)

As requirements are gathered, an overall vision of system functions and features begins to materialise, but it is difficult to move to more technical requirements without an understanding of how the system will be used. Usage scenarios – use cases – provide a description of how the system will be used from the end-user's point of view. Creating use cases forces the requirements engineering team to think about the system from the user's perspective, is invaluable when trying to elicit requirements, and helps identify actors and the interactions between them and the system.

5.3.4 Elicitation Work Products

The work products produced as a consequence of requirements elicitation vary depending on the size of the system/product to be built, but should include:

- A statement of need and feasibility.

- A bounded statement of scope for the system/product.
- A list of stakeholders who participated in requirements elicitation.
- A description of the technical environment.
- A list of requirements and the domain constraints that apply to each.
- A set of usage scenarios that provide insight into system/product use under different operating conditions.
- Any prototypes developed to better define requirements.

5.4 Developing Use Cases

A use case tells a stylised story about how an end user (actor) interacts with the system under a specific set of circumstances. An actor is not a specific person but a role that different people (or devices) can play. The story that describes how the system is used must be told from the point of view of the actor, and is often written in the language of the application domain rather than technical jargon.

Basic questions that should be answered by a use case:

- Who is the primary actor – the stakeholder who calls on the system to deliver one of its services?
- What is the actor's goal?
- What preconditions should exist before the story begins?
- What main tasks/functions are performed by the actor?
- What extensions might occur as the story unfolds?
- What variations in the actor's interaction are possible?
- What system information will the actor acquire, produce, or change?
- Will the actor inform the system about changes in the external environment?
- What information does the actor desire from the system?
- Does the actor wish to be informed about unexpected changes?

A simplified use case template for a home security system (SafeHome) example – 'access the control panel using a keypad':

Field	Description
Use Case	Access the control panel using a keypad (Basic Path)
Actors	Homeowner
Precondition	The security system has been programmed for a password and is

Field	Description
	disarmed
Description	1. Homeowner observes SafeHome control panel to determine system status. 2. If the system is not ready, homeowner secures all windows/doors so ready indicator is present. 3. Homeowner uses keypad to key in password. 4. Password is compared with correct password in system database. 5. Control panel displays 'ready' state.
Exception Paths	Sensors indicate windows/doors are open; system not ready; incorrect password entered.

5.5 Building the Requirements Model

The intent of the requirements analysis model is to provide a description of the required informational, functional, and behavioural domains for a computer-based system. The model is refined and elaborated as an outcome of collaboration between the software engineer and various stakeholders. Elements of the requirements model:

- Scenario-based elements – the system is described from the user's point of view using scenarios (use cases) and activity diagrams that provide detail about the process flow of the system.
- Class-based elements – each usage scenario implies a set of objects manipulated by an actor as it interacts with the system; these objects are categorised into classes, along with their attributes and operations. Fully described using class diagrams, CRC (class-responsibility-collaborator) models, and collaboration diagrams.
- Behavioural elements – the behaviour of a system/class can have a profound effect on the design; this behaviour must be modelled, commonly using a state diagram.
- Flow-oriented elements – information is transformed as it flows through a computer-based system; the system accepts input, applies functions to transform it, and produces output. Data flow diagrams (DFDs) depict this.

Analysis Patterns

As analysis models are created, the software team may notice that certain problems reappear across an application domain; analysis patterns suggest solutions (e.g., a class definition or a set of collaborating classes) within the application domain that can be reused when modelling similar problems for new applications. Patterns are integrated into the analysis model by reference to the pattern name and can

accelerate the creation of the model.

5.6 Negotiating Requirements

In an ideal requirements engineering context, requirements negotiation would identify the priority, cost, and risk associated with requirements and rank them accordingly. In reality, stakeholders begin to argue about which requirements have merit and which do not. The intent of negotiation is to develop a project plan that meets stakeholder needs while reflecting real-world constraints (time, resources).

Steps in a negotiation approach:

1. Identification of the key stakeholders who will be involved in the negotiation.
2. Determination of each stakeholder's 'win conditions'.
3. Negotiation of the win conditions of each stakeholder to reconcile them into a set of win-win conditions for all concerned (including the software project team).

Although a wide array of negotiation activities occur, the following guidelines can lead to more effective negotiation: recognise that it is not a competition; map out a strategy; listen actively; focus on the other party's interests; do not let it become personal; be creative; be ready to commit once agreement is reached.

5.7 Validating Requirements

As each element of the requirements model is created, it is examined for consistency, omissions, and ambiguity. The requirements represented in the model are prioritised and grouped within packages. A review of the requirements model addresses the following questions:

- Is each requirement consistent with the overall objectives for the system/product?
- Have all requirements been specified at the proper level of abstraction? Are some requirements too detailed while others too abstract?
- Is the requirement really necessary, or does it represent an add-on feature that may not be essential to the objective of the system?
- Is each requirement bounded and unambiguous?
- Does each requirement have attribution – that is, is a source (generally a specific individual) noted for each requirement?
- Do any requirements conflict with other requirements?
- Is each requirement achievable in the technical environment that will house the system/product?
- Is each requirement testable, once implemented?

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

- Does the requirements model properly reflect the information, function, and behaviour of the system to be built?
- Has the requirements model been 'partitioned' in a way that exposes progressively more detailed information about the system?
- Have requirements patterns been used to simplify the requirements model? Have all patterns been properly validated? Are all patterns consistent with customer requirements?

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

Chapter 6: Requirements Modeling – Scenarios, Information and Analysis Classes

6.1 Requirements Analysis

Requirements analysis results in the specification of the software's operational characteristics, indicates the software's interface with other system elements, and establishes constraints that the software must meet. It allows the software engineer to elaborate on basic requirements established during earlier requirements engineering tasks, and to build models that depict user scenarios, functional activities, problem classes and their relationships, system and class behaviour, and the flow of data as it is transformed.

6.1.1 Overall Objectives and Philosophy

Regardless of the analysis method used, every analysis model should achieve three primary objectives:

- To describe what the customer requires.
- To establish a basis for the creation of a software design.
- To define a set of requirements that can be validated once the software is built.

The analysis model bridges the gap between a system-level description that describes overall system functionality and a software design that describes the software's application architecture, user interface, and component-level structure.

6.1.2 Analysis Rules of Thumb

- The model should focus on requirements that are visible within the problem or business domain; the level of abstraction should be relatively high.
- Each element of the analysis model should add to an overall understanding of software requirements and provide insight into the information domain, function, and behaviour of the system.
- Delay consideration of infrastructure and other non-functional models until design.
- Minimise coupling throughout the system.
- Be certain that the analysis model provides value to all stakeholders.
- Keep the model as simple as it can be, given the description of the problem being solved.

6.1.3 Domain Analysis

Object-oriented domain analysis is the identification, analysis, and specification of common, reusable capabilities within a specific application domain (e.g., healthcare, banking). The intent is to find or create classes that are broadly applicable so that they may be reused. Domain analysis may be viewed as

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

an umbrella activity for the software process; results are presented to software engineering teams working on projects within the domain, and reusable analysis classes are placed in a repository so that they may be reused for future analysis models.

6.1.4 Requirements Modeling Approaches

- Structured analysis – considers data and the processes that transform the data as separate entities; data objects are modelled to define their attributes and relationships, and processes that transform data objects are modelled showing how they transform data as it flows through the system (e.g., via data flow diagrams).
- Object-oriented analysis – focuses on the definition of classes and the manner in which they collaborate with one another to satisfy customer requirements; UML and the Unified Process are widely used object-oriented approaches today.

6.2 Scenario-Based Modeling

Requirements modeling using scenario-based techniques is often the first step for many software teams. Use cases (introduced in Chapter 5) are the primary requirements modeling technique for many object-oriented and traditional systems.

6.2.1 Creating a Preliminary Use Case

Recall that use cases capture the interaction between actors and the system in a specific context (a goal the actor wants the system to achieve). To develop an effective set of use cases: identify the actors, determine what each actor requires or requests as the system performs its work, and create a preliminary use case for each distinct actor-goal interaction.

A preliminary use case is written in narrative form. If further elaboration is required, additional information is added by expanding the narrative or by moving to a template-based approach that specifies preconditions, actor's actions, system responses, and post-conditions.

6.2.2 Refining a Preliminary Use Case

Once the preliminary use case is developed, it is refined by breaking narrative text into discrete steps, showing exactly what actions are taken and how the system responds. Alternative interaction (exception handling) is added, and refinement continues until the use case appears to be complete.

6.2.3 Writing a Formal Use Case

A formal use case template presents structured detail: use-case name, primary actor, goal in context, preconditions, trigger, scenario (numbered actor actions and system responses), exceptions, priority, when available, frequency of use, secondary actors, secondary actor's goals, and miscellaneous

notes/issues. This formal structure is especially useful when use cases will be handed off to a design team, used for test-case derivation, or when regulatory documentation is required.

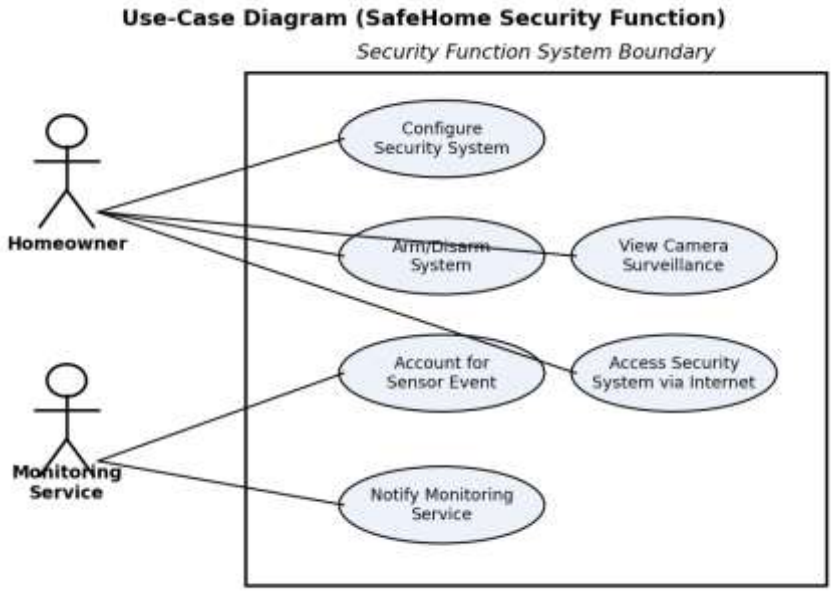


Fig. 6.1 – A use-case diagram showing actors and the use cases of the security-monitoring system (illustrative example)

6.3 UML Models That Supplement the Use Case

Text-based use cases must ultimately be supplemented by one or more diagrams that add rigour, provide additional insight into the interaction, and depict the process flow associated with the use case. UML provides a number of these diagrams.

6.3.1 Developing an Activity Diagram

The UML activity diagram supplements the use case by providing a graphical representation of the flow of interaction within a specific scenario. Similar to a flowchart, the activity diagram uses rounded rectangles to denote a specific system function, arrows to represent flow through the system, decision diamonds to represent a branching decision, and solid horizontal lines to indicate that parallel activities are occurring.

6.3.2 Swimlane Diagrams

A swimlane diagram is a useful variation of the activity diagram; it allows the modeller to represent the flow of activities described by the use case and, at the same time, indicate which actor (if there are multiple actors involved in a specific use case) or analysis class has responsibility for the action described by an activity rectangle. Responsibilities are represented as parallel line segments that divide

the diagram vertically or horizontally, dividing the diagram into 'lanes'.

6.4 Data Modeling Concepts

If information within the software application domain is complex or if the primary function of the application is to create, store, manage, and manipulate large data stores, data modeling is used. Data modeling defines all data objects processed within the system, the relationships between the data objects, the associated attributes, and the association between data objects and the processes that transform them.

6.4.1 Data Objects, Attributes, and Relationships

- Data object – a representation of composite information that must be understood by the software (e.g., a person, place, event, or a description). A data object encapsulates only data – no reference to operations that act on the data is made. A data object can be an external entity, a thing, an occurrence/event, a role, an organisational unit, a place, or a structure.
- Attributes – define the properties of a data object and take on one of three characteristics: they can be used to name an instance of the data object, describe the instance, or make reference to another instance in another table. One or more of the attributes must be defined as an identifier – a 'key' that can be used to find, sort or identify an instance of the data object.
- Relationships – data objects are connected to one another in different ways; consider two data objects, Book and Bookstore – the relationships 'is stocked in', 'is ordered by', 'is delivered to' connect these objects.

6.4.2 Cardinality and Modality

The data model must be capable of representing the number of occurrences of objects in a given relationship – cardinality – which is the specification of the number of occurrences of one [object] that can be related to the number of occurrences of another. Cardinality is usually expressed as 'one' or 'many' (e.g., one-to-one, one-to-many, many-to-many).

Modality of a relationship is 0 if there is no explicit need for the relationship to occur, or the relationship is optional. The modality is 1 if an occurrence of the relationship is mandatory.

6.4.3 Entity-Relationship Diagram (ERD)

The entity-relationship diagram (ERD), originally proposed by Peter Chen, focuses solely on data (and therefore is relevant to database design and business-process re-engineering), depicting relationships between data objects using entity boxes, relationship diamonds/labels, and connecting lines annotated with cardinality and modality.

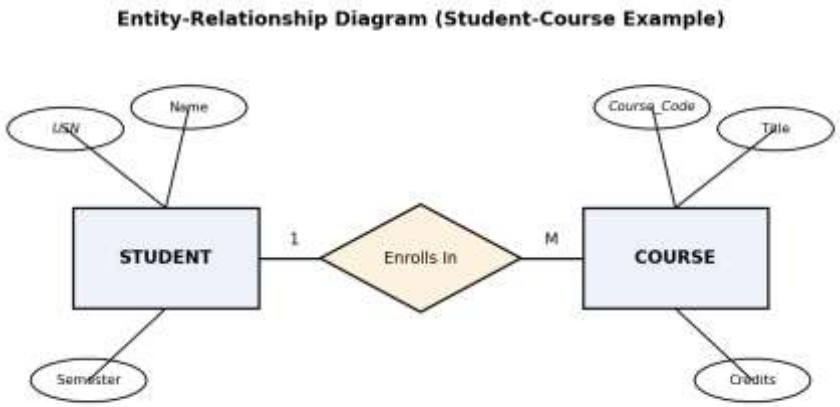


Fig. 6.2 – Entity-relationship diagram illustrating entities, a relationship, attributes, and cardinality

6.5 Class-Based Modeling

Class-based modeling represents the objects that a system will manipulate, the operations (also called methods or services) that will be applied to the objects to effect the manipulation, relationships between the objects, and the collaboration that occurs between classes that are defined as part of the problem to be solved.

6.5.1 Identifying Analysis Classes

Grammatical parsing is used to isolate potential classes from a processing narrative or use case; classes are typically manifested in one of the following ways within the narrative:

- External entities that produce or consume information to be used by the system.
- Things that are part of the information domain for the problem.
- Occurrences or events that happen within the context of system operation.
- Roles played by people who interact with the system.
- Organisational units relevant to the application.
- Places that establish the context of the problem and the overall function of the system.
- Structures that define a class of objects or related classes of objects.

A potential analysis class is a viable candidate for use in the analysis model if it satisfies the following characteristics, proposed by Coad and Yourdon:

1. Retained information – information about the potential class must be remembered so that the system can function.

2. Needed services – the potential class must have a set of identifiable operations that can change the value of its attributes in some way.
3. Multiple attributes – a class with a single attribute may, in fact, be useful during design but is probably better represented as an attribute of another class during analysis.
4. Common attributes – multiple attributes can be defined for a potential class, and these attributes apply to all instances of the class.
5. Common operations – common operations can be defined for a potential class, and these operations apply to all instances of the class.
6. Essential requirements – external entities that appear in the problem space, and produce or consume information essential to the operation of any solution for the problem, will almost always be defined as classes in the requirements model.

6.5.2 Specifying Attributes

Attributes describe a class that has been selected for inclusion in the requirements model, and to a great extent, define the class in the context of the problem at hand. They are determined by understanding the problem and choosing those things that make sense for the given class within the context of the problem domain.

6.5.3 Defining Operations

Operations define the behaviour of an object and can be divided into four broad categories:

- Operations that manipulate data in some way (e.g., adding, deleting, reformatting, selecting).
- Operations that perform a computation.
- Operations that inquire about the state of an object.
- Operations that monitor an object for the occurrence of a controlling event.

As with attributes, grammatical parsing of the processing narrative for the software is used to determine appropriate operations for a class: the verbs used within the narrative correlate with an operation, and the object of the verb is often the attribute/class on which the operation works.

6.5.4 Class-Responsibility-Collaborator (CRC) Modeling

CRC modeling provides a simple means for identifying and organising the classes that are relevant to system/product requirements. A CRC model is a collection of standard index cards (or their virtual equivalent) that represent classes. The cards are divided into three sections: the class name in the top; class responsibilities on the left; and collaborators on the right.

- Class – represents a collection of similar objects; may be one of: entity classes (also called model or business classes), boundary classes (used to create the interface – e.g., screens, printed reports), or controller classes that manage a 'unit of work' from start to finish, i.e., control other objects associated with a use case.
- Responsibilities – the attributes and operations that are relevant for the class; stated as broadly as possible and, taken together, describe the class comprehensively within the context of the problem.
- Collaborators – the classes that are required to provide a class with the information needed to complete a responsibility. In general, a collaboration implies either a request for information or a request for some action.

A simplified CRC card for the class 'Sensor' in a security-monitoring system:

Class: Sensor	
Responsibilities	Collaborators
Recognises abnormal condition Sends signal to Control Panel Tests self-status	Control Panel

6.5.5 Associations and Dependencies

Two analysis classes are often related to one another in some fashion, represented in class diagrams as an association. Associations can be refined by indicating multiplicity (the cardinality between classes, similar to data modeling). When two classes are related, and the relationship indicates that changes to one class will affect the other, a dependency exists, represented with a dashed arrow in UML.

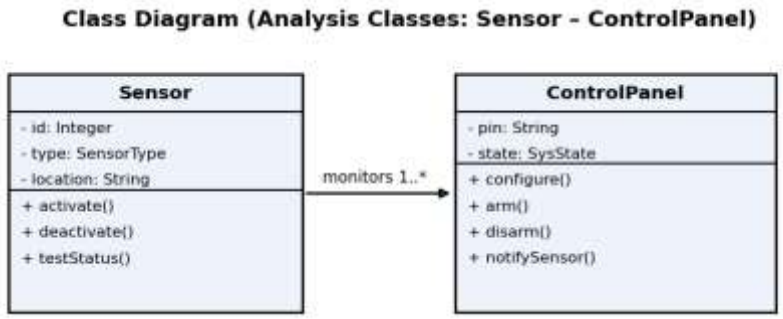


Fig. 6.3 – A UML class diagram showing two analysis classes and an association with multiplicity

6.5.6 Analysis Packages

An important part of the requirements model is the categorisation of different elements. Requirements are categorised and related to one another; each category becomes a 'package' – a mechanism for organising elements of the requirements model into groups. Packages are annotated with the class names that they contain, and can be a useful way to organise the requirements model into manageable pieces, especially for large and complex systems.

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

Chapter 7: Requirements Modeling – Flow, Behaviour, Patterns and WebApps

(As per syllabus scope, only Sections 7.1 to 7.3 – Requirement Modeling Strategies: Flow-Oriented Modeling and Behavioral Modeling – are covered in this module.)

7.1 Requirements Modeling Strategies – Introduction

Over the years, many methods have been proposed to translate an unbounded statement of software scope into a working software product. Each provides a unique method for creating a robust model of the information, functional, and behavioural domains as a basis for software design. Flow-oriented modeling was the first widely used technique and remains a viable modeling target when the primary interest is the flow of data through a system and the transforms applied to it.

7.2 Flow-Oriented Modeling

Flow-oriented modeling represents how data objects are transformed by processing functions. Every computer-based system is essentially an information transform – whether embedded (converting sensor readings to display information) or an enterprise information system (transforming a business query into corresponding data to satisfy the request). A data flow diagram (DFD) is the diagrammatic form used to represent this transformation.

7.2.1 The Data Flow Diagram (DFD)

The DFD enables the software engineer to develop models of the information domain and functional domain at the same time. As the DFD is refined into greater levels of detail, the analyst performs an implicit functional decomposition of the system. At the same time, the DFD refinement results in a corresponding refinement of data as it moves through the process that embodies the application.

A DFD is composed of four principal notational elements:

- External entity (or terminator) – a producer or consumer of data, represented as a rectangle; lies outside the bounds of the system to be modelled and represents another system, a device, a person.
- Process (or transform) – represented as a rounded rectangle or circle; transforms information as it flows through it (e.g., accepts input, does something to it, and produces output).
- Data flow – represented by an arrow; represents one or more data items that flow between the external entity, process, and data store.
- Data store – represented by two parallel lines (or an open-ended rectangle); a repository of data that is to be stored for use by one or more processes.

The context-level DFD (level 0) depicts the entire software element as a single bubble with input and output data indicated by incoming and outgoing arrows respectively; additional processes and

information flow paths are represented as the level-0 DFD is partitioned to reveal more detail (level 1, level 2 and so on), a process called levelling or partitioning.

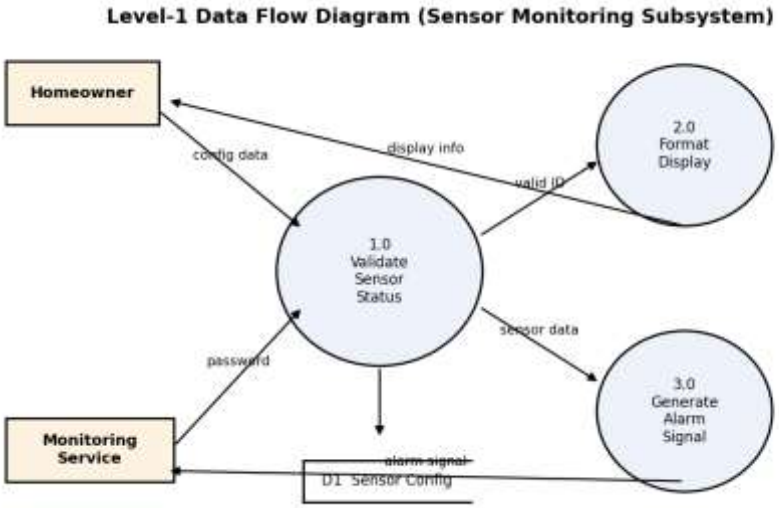


Fig. 7.1 – Level-1 data flow diagram illustrating external entities, processes, data flows, and a data store

7.2.2 Control Flow and Control Specification

For many types of applications, data alone does not provide sufficient insight into the workings of a process, so a control-flow model is created as a supplement. A control flow model provides additional insight into control flow – events that cause the system to transition from one state (e.g., 'weapon safe') to another (e.g., 'weapon armed'). A control specification (CSPEC) represents the behaviour of the system in two different ways: it contains a state diagram that is a sequential specification of behaviour, and it may also contain a program activation table (PAT) – a combinatorial specification of behaviour.

7.3 Creating a Behavioral Model

The behavioural model indicates how software will respond to external events or stimuli, and is created by performing the following steps: evaluate all use cases to fully understand the sequence of interaction within the system; identify events that drive the interaction sequence and understand how these events relate to specific objects/classes; create a sequence for each use case; build a state diagram for the system; and review the behavioural model to verify accuracy and consistency.

7.3.1 Identifying Events with the Use Case

A use case represents a sequence of activities that involves actors and the system. In general, an event occurs whenever the system and an actor exchange information; an event is not the information that is exchanged but rather the fact that information has been exchanged. Once all events have been identified,

they are allocated to the objects involved, and objects may be the sender or receiver of an event.

7.3.2 State Representations – State Diagrams

Two different characterisations of states must be considered when a behavioural model is created:

- The state of each class as the system performs its function.
- The state of the system as observed from the outside as the system performs its function.

The state of a class takes on both passive and active characteristics: a passive state is simply the current status of all of an object's attributes; an active state indicates the current status of the object as it undergoes a continuing transformation or processing. A state diagram for a class depicts the sequence of states through which a class passes during its life, the events that cause the transitions from state to state, and the actions taken by the object as a consequence of a particular event.

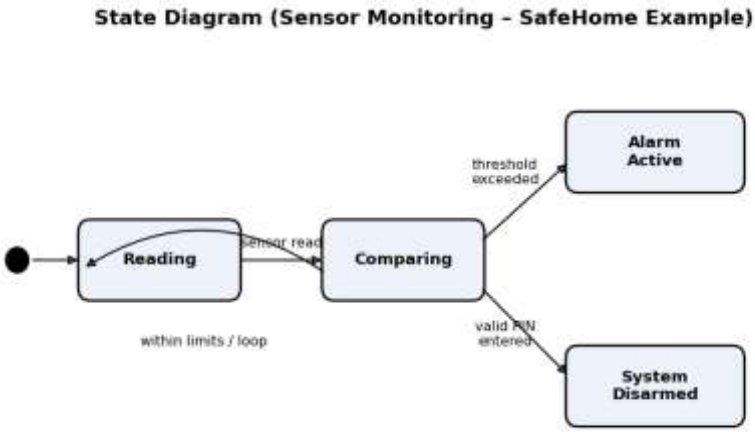


Fig. 7.2 – State diagram showing states, transitions with guard/event labels, and initial state

A state diagram for the system indicates how the system responds to external events, and is created by examining all classes and determining how each state (of each class) is affected by the outside world. The behaviour of the system as a whole can be represented by a sequence diagram (a UML modeling notation used primarily to show interactions between actors and the system during a use case) or a state diagram that merges the states of the classes that were defined earlier.

Previous-Year VTU Examination Questions (Module 2 Pattern)

The following questions are representative of the VTU question pattern typically set on this module's content and are provided for revision purposes. Marks distribution follows the usual 10-mark full-question / 5–6-mark sub-question CIE-SEE pattern; verify against the current university question paper before an examination.

Unit-style Question Bank

1. Explain the requirements engineering process with its seven distinct tasks. (10 Marks)
2. What is a stakeholder? Explain the importance of identifying stakeholders and recognising multiple viewpoints during requirements engineering. (6 Marks)
3. Describe the guidelines followed in collaborative requirements gathering. (6 Marks)
4. Explain Quality Function Deployment (QFD) and the three types of requirements it identifies, with examples. (10 Marks)
5. What is a use case? Explain the basic questions that should be answered by every use case. (8 Marks)
6. Develop a use case for any system of your choice (e.g., withdrawing cash from an ATM / SafeHome security system) showing actors, preconditions, description and exception paths. (10 Marks)
7. Discuss the elements of the requirements model with a neat diagram. (8 Marks)
8. Explain the steps involved in requirements negotiation. (6 Marks)
9. List and explain the questions that must be answered while validating a requirements model. (8 Marks)
10. What are the objectives and rules of thumb followed in requirements analysis? (8 Marks)
11. Differentiate between structured analysis and object-oriented analysis approaches to requirements modeling. (6 Marks)
12. Explain how a preliminary use case is refined into a formal use case, with an example. (10 Marks)
13. What is a UML activity diagram? Explain with reference to swimlane diagrams. (8 Marks)
14. Explain the basic data modeling concepts – data objects, attributes, and relationships – with an example. (8 Marks)
15. What are cardinality and modality? Explain with an entity-relationship diagram. (10 Marks)
16. Explain the process of identifying analysis classes from a processing narrative, listing the characteristics proposed by Coad and Yourdon. (10 Marks)

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106

17. What is CRC modeling? Explain class, responsibilities, and collaborators with a sample CRC card. (8 Marks)
18. Differentiate between association and dependency in class-based modeling. (5 Marks)
19. Explain analysis packages and their role in organising the requirements model. (5 Marks)
20. What is a Data Flow Diagram (DFD)? Explain its notational elements with a context-level and level-1 example. (10 Marks)
21. Explain control flow modeling and the role of a control specification (CSPEC). (6 Marks)
22. Explain the steps involved in creating a behavioural model for a system. (8 Marks)
23. What is a state diagram? Differentiate between passive and active states of a class, with an example. (10 Marks)
24. Explain how events are identified from a use case and allocated to objects during behavioural modeling. (6 Marks)

Prepared by: Kavya.K.M, Assistant Professor, Dept of CSE(IC), Sri Sairam College of Engineering, Anekal, Bengaluru, Karnataka, India - 562106